

CCAR-P Domain 1 Practice Workbook

Solution Design and Architecture. 17 percent of the exam, about 11 of 63 questions. Six objectives, with 1.3 (pattern selection) and 1.5 (decomposition) carrying the most weight.

What this workbook is

A set of exercises built on the Kaveri Freight control tower and the eight runnable exhibits in **d1/**. It does not repeat the forty questions in the Control Tower Brief. Everything here is new material in the same style, so you can sit it after the Brief's drill and measure whether the lessons stuck.

How to use it

Section	What you do	Time	Objective
1 Recall sheets	Fill the blanks from memory, no notes. Check against the Brief afterwards.	20 min	all
2 Tell-spotting drill	Fourteen one-line stems. Underline the tell, name the pattern.	15 min	1.3, 1.5
3 Predict the output	Eight traces from the d1/live code. Write what prints before you run it.	25 min	1.3, 1.5
4 Red pen	Eight code snippets. Name every defect and the law it breaks.	25 min	1.3, 1.5
5 Arithmetic	Ten numeric problems. Cost, latency, budgets, SLA tails.	25 min	1.3, 1.6
6 Architecture completion	Five described architectures, each missing one arc.	15 min	1.2
7 Multi-agent	Six short-answer items on topology, isolation and SDK caps.	15 min	1.4
8 Pillar matrix	Score three options against five numbered pillars.	15 min	1.6
9 Twenty-eight questions	Exam-style, one sitting, 110 seconds each, key sealed.	55 min	all
10 Sealed key	Answers with the tell underlined and every wrong option tagged by family.		
11 Miss ledger	One row per miss: objective, family, the tell you skipped.	10 min	

Rules of the sitting

1. Underline the deciding clause in the stem before you read any option.
2. Mark, do not dwell. 110 seconds per question in section 9.
3. The key stays sealed until a section is finished.
4. Every miss gets a ledger row with a family code. The family is the lesson, not the question.

Distractor families used in the key

Code	Family	What it looks like in D1
OE	Over-engineered	The most capable architecture in the list, chosen when the stem already fixed a simpler one. The magnificent bridge.
TR	Trade-off reversed	Optimises a pillar the stem did not fix, or refuses the agent when the stem said the path is unknowable.
CM	Constraint missed	Ignores the latency budget, the cost ceiling, the regulated data, or the real-time requirement.
HL	Human loop skipped	Removes or never adds the gate on money, law or people.
VG	Vocabulary gap	Uses a term for something it is not. Calls a bloated prompt 'routing', calls a code-written list 'orchestration'.
SF	Silent failure	Code that swallows a stop_reason, loops without a budget, or returns partial text as done.

Code	Family	What it looks like in D1
SD	Sounds diligent	An option that is busy and worthy but decides nothing about Kaveri's own workload.

The five laws, for reference while you work

Law	Statement
1	The constraint picks the pattern, not the ambition.
2	Workflows when the path is known; agents when it is not.
3	Every loop needs a budget and an exit.
4	Decompose on discovery, not in advance.
5	Value is a pillar with a number attached.

Section 1 Recall sheets

No notes. Fill every blank, then check against the Brief and the d1/README. A blank you cannot fill is a ledger row.

1a The seven patterns and their tells

Pattern	Exhibit	The stem says (the tell)	Who owns the path
Augmented LLM	E1		
Prompt chaining	E2		
Routing	E3		
Parallel: sectioning	E4		
Parallel: voting	E4		
Orchestrator-workers	E5		
Evaluator-optimizer	E6		
Agent loop	E7		
Re-planning	E8		

1b The stop_reason values and what the loop must do with each

Value	Meaning	Correct handling in E7	Wrong handling that stems love
end_turn			
tool_use			
max_tokens			
stop_sequence			
pause_turn			
refusal			
(context window exceeded)			

1c The five pillars with Kaveri's numbers

Pillar	Kaveri target (number + horizon)	Design choice it justifies
Efficiency		
Cost		
Productivity		
Performance / SLA		
Transformation		

1d The two organs of every loop, and where they sit in each exhibit

Exhibit	The budget is called	The exit is called	The floor beneath both
E6 evaluator-optimizer			
E7 agent loop			
E8 re-planning			(two budgets: name both)

1e Multi-agent caps to quote (current SDK)

Cap	Default or behaviour
Spawn depth	
Concurrency	
Spend cap (name the parameter and its three effects)	

1f The four stages of the reference architecture, and the two arcs the wrong options delete

Stage	What lives there at Kaveri
Input	
Processing	
Output	
Feedback	
Arc 1 that stems delete	
Arc 2 that stems delete	

Section 2 Tell-spotting drill

Fourteen stems, one line each. For each: underline the tell, write the pattern (or 'not an LLM seat' / 'plain code' / 'gate'), and name the family of the most tempting wrong answer. Do all fourteen before opening the key.

#	Stem	Pattern	Tempting wrong answer, family
1	Every damage claim follows the same three steps: extract fields, check the policy clause, draft the response. Accuracy matters more than speed.		
2	A dispatcher types a consignment id and wants a one-line ETA back. Nothing else.		
3	Exceptions arrive in six known categories, and each category needs a different tone, a different SOP and a different model		
4	Free carriers must be checked for a lane. The checks do not depend on each other. The SLA is 30 seconds.		
5	Refund decisions above Rs 10,000 are rare, high-stakes, and the desk accepts three times the cost for extra confidence.		
6	Each customs investigation depends on what the previous lookup reveals; some need two lookups, some need seven;		
7	From memory, the subtasks can be listed up front, but the list differs for each brief, and the delegate-then-synthesise shape never changes.		
8	A reply that improves measurably against a written three-item rubric, and the desk wants at most two revision rounds.		
9	Halfway through a planned investigation a worker discovers the hold is a customs issue, which makes two of the remaining planned subtasks pointless.		
10	Reamer equals Paskart and status equals RTO, re-book via BlueDart. Always.		
11	The category is already a field on the ticket, typed by the dispatcher who opened it.		
12	The desk wants one prompt that contains the refund instructions, the customs instructions and the delay instructions, and lets the model pick which to follow.		
13	A solution bot needs a divert decision every 40 milliseconds from structured scanner data.		
14	Dispatchers resolve ambiguous address exceptions by phoning the customer, and 38 percent end with information the ticket never contained.		

Section 3 Predict the output

Each item shows a fragment of the d1/live code or a scenario for the dry-run exhibits. Write what prints, line by line, then run the file to check. The point is not the wording; it is the `stop_reason` trace and the control flow.

3.1 E1 on battery

Run: `DRY_RUN=1 python d1/e1_augmented_llm.py`. Write the four lines it prints. Then state which line would change if the model had answered without the tool, and what the category would most likely have been.

3.2 E2's gate

Desk 1 returns the string 'Delay.' with a capital letter and a full stop. `classify()` applies `.lower()` only. What does `gate()` do, and is that behaviour a bug or the gate doing its job? What one-line change in `classify()` would let the line continue?

3.3 E4 timing

Three scouts take 1.8 s, 2.4 s and 1.1 s. Write the two wall-clock numbers E4 prints for sectioning and sequential, and the seconds saved.

3.4 E6 rounds

`MAX_ROUNDS = 2`. The editor says PASS on round 1. How many model calls were made in total, counting the first draft? Now the editor criticises on round 1 and round 2. How many calls, and what is returned?

3.5 E7 default

Write the E7 trace for scenario default: one line per turn with its stop_reason and tool name. How many paid calls? What is the maximum E7 could ever make with BUDGET = 6?

3.6 E7 truncate

Scenario truncate. Write the trace from turn 3 onward and say what happens to the text that arrived with max_tokens. What would a loop that treated max_tokens as finished have returned?

3.7 E7 exhaust

Scenario exhaust: the stand-in asks for get_shipment_status on every turn. Write the final printed line exactly. What is the one word that makes this line acceptable to Meera rather than a failure?

3.8 E8 replan

Initial plan: 1 status check, 2 carrier options, 3 re-route quote. After run 1 the foreman says ok false, drop [3], add ['file e-way bill', 'notify Padma of revised ETA']. MAX_NEW = 3. Write the order of 'run' lines that follow, the ids of the two new tasks, and the final 'grew' line. What changes if MAX_NEW = 1?

Section 4 Red pen: find every defect

Each snippet is wrong in one to three ways. Name each defect, the law it breaks, and the one-line fix. A defect you cannot name is a ledger row under SF or VG.

4.1

```
while True:
    r = client.messages.create(model=MODEL, max_tokens=1024, tools=TOOLS, messages=msgs)
    if r.stop_reason == "tool_use":
        run_tools(r, msgs)
    else:
        return r.content[0].text
```

Defect	Law broken	One-line fix

4.2

```
for turn in range(BUDGET):
    r = client.messages.create(...)
    if r.stop_reason == "tool_use":
        ...
    elif r.stop_reason == "end_turn":
        return text_of(r)
    elif r.stop_reason == "refusal":
        msgs.append({"role": "user", "content": "Please try again."})
    else:
        return text_of(r)
```

Defect	Law broken	One-line fix

4.3

```
call = next(b for b in msg.content if b.type == "tool_use")
result = get_shipment_status(call.input["consignment_id"])
msgs.append({"role": "user", "content": [{"type": "tool_result", "content": json.dumps(result)}]})
msg = client.messages.create(..., messages=msgs)
```

Defect	Law broken	One-line fix

4.4

```
def replan(brief):
    tasks = plan_from(brief)
    while tasks:
        sub = tasks.pop(0); findings[sub["id"]] = run_worker(sub, findings)
        v = ask_json(MODEL, "Does the plan still hold? ..." + json.dumps({"finding": findings[sub["id"]], "remain": findings[sub["id"]]}))
        if not v["ok"]:
            tasks += v["add"]
```

Defect	Law broken	One-line fix

4.5

```
SPECIALISTS = {"refund": (MODEL_BIG, ...), "customs": (MODEL_MID, ...), "delay": (MODEL_SMALL, ...)}
cat = classify(email)
model, system = SPECIALISTS[cat]
return ask(model, system + email)
```

Defect	Law broken	One-line fix

4.6

```
verdicts = await asyncio.gather(*[ask_async(MODEL_BIG, "APPROVE or REJECT only. Delay apology:\n" + apology) for apology in apologies])
return max(set(verdicts), key=verdicts.count)
```

Defect	Law broken	One-line fix

4.7

```
def run_worker(sub, findings):
    return ask(MODEL_SMALL, f"Subtask: {sub['task']}\nEverything so far: {json.dumps(findings)}\nFull brief: {BRIEF}")
```

Defect	Law broken	One-line fix

4.8

draft = ask(MODEL, "Reply to this email well, check your own tone and length, and fix anything wrong before answer
return draft

Defect	Law broken	One-line fix

Section 5 Arithmetic

Ten problems. Show the working; the exam's numeric stems reward the candidate who can reproduce the number, not recognise it. Rs is rupees.

5.1 Tiered average cost

Kaveri sees 1,200 exceptions a day. Under the tiered design, 80 percent go through router plus chain at Rs 1.50 each, 15 percent through the customs explanation ward at Rs 6.00, and 5 percent through the agent tier at Rs 28.00. Compute the average cost per exception and the daily spend. Does it meet the cost pillar of at most Rs 6 per exception?

5.2 The one-big-agent design

Same 1,200 a day, but every ticket enters the agent loop at an average Rs 22.00. Average cost, daily spend, and the daily difference against 5.1. Which family is the person who chose this design?

5.3 Parallel latency

Three scouts take 1.8 s, 2.4 s and 1.1 s. Wall clock for sectioning, wall clock for sequential, seconds saved. Does sectioning change the token bill?

5.4 Voting on the right thing

A single judge call costs Rs 4. Refunds are 9 percent of 1,200 tickets. Daily cost of one judge on refunds versus three judges. Now someone proposes voting on delay apologies too (55 percent of volume, Rs 0.80 per call). Daily extra cost of that, and the pillar it breaches.

5.5 The budget as a price cap

BUDGET = 6 and the average agent call costs Rs 3.20. Worst-case cost of one ticket. Now a while True loop on a ticket whose tool keeps returning the same hold, one call every 2 seconds, unnoticed for an hour. Cost of that hour.

5.6 SLA tail

The SLA is 99 percent of drafts within 30 s. Of 1,200 daily drafts, the agent tier handles 5 percent, and every agent-tier draft takes longer than 30 s. What percentage lands within 30 s? Is the SLA met? What is the largest share the agent tier may take if all other drafts are fast?

5.7 Re-planning cost ceiling

An initial plan has 3 tasks. MAX_NEW = 3. Each worker call costs Rs 2 and each check_plan call costs Rs 4; the plan call and the synthesis call cost Rs 4 each. Maximum number of tasks that can run, and the maximum cost of one replan() run. What is the maximum without the cap?

5.8 Productivity pillar

A dispatcher resolves 30 exceptions a day when drafting from scratch. With draft-plus-edit the pillar target is 3 times throughput. New per-dispatcher rate, total capacity for 40 dispatchers, and headroom against 1,200 a day.

5.9 Concurrency cap

An orchestrator wants 50 carrier-check subagents at once; each takes 6 s. Concurrency default is 20. Minimum wall clock for all 50, and the number of waves.

5.10 Spawn depth

Spawn depth default is 3. The orchestrator is layer 1 and spawns a customs subagent (layer 2), which spawns a document-checker (layer 3), which tries to spawn a translator. What happens, and which law is this cap an instance of?

Section 6 Architecture completion

Each architecture is described in words, as a 1.2 stem would. Each is complete except for one arc or one stage. Name what is missing, which stage it belongs to, and the family of the person who left it out.

#	Architecture as described	Missing arc or stage	Stage	Family
1	Email, portal and phone transcripts each feed their own prompt. A router sends each ticket to a specialist chain.			
2	Drafts go to dispatchers. Dispatcher edits are diffed weekly into an oval. A router sorts into six wards. Refund replies are			
3	Normaliser, router, specialist chains, parallel carrier checks, are			
4	dispatcher, a human gate, agent tier, drafts, gate, a weekly			
5	Normaliser, router, chains, agent tier, drafts, gate, feedback			

Section 7 Multi-agent short answers

Two or three sentences each. Quote a cap where one applies.

7.1 Three subagents (customs, carrier, customer) each turn out to need every other's field notes to finish. Which topology property has stopped paying, and what would you do?

7.2 A triage shift hands a case to a resolution shift with a free-text note. The resolution agent re-asks the customer for the consignment id and re-runs the authorization check. Name the defect and what the baton should contain.

7.3 Meera asks why the orchestrator's own context stays small when it has launched six subagents over ten minutes. Explain in terms of what travels back.

7.4 A per-query max_budget_usd is hit while two background subagents are still running and a seventh spawn is requested. List the three things that happen.

7.5 The single-agent design gives one loop all nine tools and one prompt covering refunds, customs and delays. Name two symptoms that say it is time to split, and the law behind the split.

7.6 A stem offers 'peer handoff' and 'orchestrator plus subagents' for a process with two clear phases, triage then resolution, where phase two never needs phase one's raw notes. Which fits and why?

Section 8 The pillar matrix

Three candidate designs for a second site, Kaveri Bengaluru, 800 exceptions a day. Fill every cell with a number or 'n/a', mark each cell met or breached against the target, and then answer the three questions. An option with a cell you cannot fill is not a candidate.

Pillar and target	D One agent for everything (avg Rs 24, p50 draft 38 s, first touch 3 min)	E Tiered: router + chains, agent for the 4 percent tangled (avg Rs 3.90, 99.2 percent of drafts under 30 s, first touch 1 min)	F Rules only, no model (Rs 0, first touch unchanged at 4 h, no drafts)
Efficiency: first touch at most 5 min			
Cost: at most Rs 6 per exception			
Productivity: 3 times dispatcher throughput			
Performance: 99 percent of drafts within 30 s			
Transformation: a new service line from patterns			

8.1 Which option is aligned, and what is the one-sentence rule that decides it?

8.2 A fourth option G promises 'a future-proof platform that improves customer experience'. Why is G not scored, and which trap is this?

8.3 A board member prefers D because 'it is the most capable'. Write the two-line reply Arjun should give, with numbers.

Section 9 Twenty-eight questions

One sitting. 110 seconds each. Underline the deciding clause before reading options. Four per objective, six each for 1.3 and 1.5. Mark your answer in the margin; the key is in section 10.

Q1 objective 1.1 Meera asks to 'automate customs holds end to end'. The log shows 70 percent of holds are cleared by re-filing the e-way bill, which can be detected from a single ticket field, and 30 percent need a written request to a customs officer. Best translation of the problem?

- A. Build an agent that handles every customs hold with tools for both paths
- B. Code handles the 70 percent rule; Claude drafts the officer request for the 30 percent; a dispatcher sends it
- C. Use the biggest model for all holds so the 30 percent are handled well
- D. Exclude customs holds from the system until the 30 percent can be fully automated

Q2 objective 1.1 Two asks arrive. One: 'summarise each phone transcript into the ticket schema'. Two: 'decide whether to re-route the consignment and issue the order'. Which reading is exam-grade?

- A. Both are Claude-shaped; the second needs a human gate because the re-route is a paid, hard-to-reverse action
- B. Only the first is Claude-shaped; the second should be a deterministic rule
- C. Both should be fully automated; a 97 percent eval score removes the need for a gate
- D. Neither is Claude-shaped until a multi-agent system is designed

Q3 objective 1.1 Which pilot statement is exam-grade for 1.1?

- A. Show the board that agentic AI can transform the exception desk
- B. Cut first touch on delay notifications from 4 hours to under 5 minutes within 4 weeks, at under Rs 2 per ticket, measured on 55 percent of daily volume
- C. Deploy Claude across refunds, customs and delays simultaneously to maximise learning
- D. Benchmark three model tiers on a sample of tickets

Q4 objective 1.1 The dispatcher who opens a ticket already types its category into a required field. The design proposes a router call that re-classifies every ticket. The correct reading?

- A. Keep the router; model classification is more reliable than a human field
- B. Drop the router call; read the field. A model re-deriving a known column is over-engineering
- C. Run both and vote
- D. Route only tickets where the field is blank, using the biggest model

Q5 objective 1.2 An architecture reads: normaliser, router, specialist chains, parallel carrier checks, agent tier with a budget, replies auto-sent for all refund amounts, outcome log, weekly eval set. What is missing?

- A. A second router to verify the first
- B. A human gate at output for refunds above the threshold; auto-send is the wrong output for an irreversible high-stakes action
- C. Caching on the specialist prompts
- D. A bigger model in the agent tier

Q6 objective 1.2 Which pair of signals belongs to the feedback stage rather than the processing stage?

- A. Router accuracy and mean tokens per call
- B. p99 API latency and cache hit rate
- C. Dispatcher edit distance on drafts and re-open rate of closed exceptions
- D. Tool-call error rate and refusal rate

Q7 objective 1.2 Phone transcripts produce 12 percent malformed tickets while email produces 1 percent. All three channels share one pipeline. Where is the 1.2-correct fix?

- A. In each specialist prompt, with instructions to tolerate malformed input
- B. At the normaliser, where transcripts are turned into the ticket schema
- C. In the agent tier, which can adapt to messy input
- D. Drop phone transcripts from automation

Q8 objective 1.2 The router labels 20 percent of tickets 'other'. One option sends 'other' to the biggest model. The architectural reading?

- A. Correct; the biggest model handles ambiguity best
- B. 'Other' at 20 percent is a signal that the category set is wrong; route those tickets to a human queue, count them, and feed them into the weekly review to add categories
- C. Raise the router's temperature so it commits to a category
- D. Retry classification three times, then drop the ticket

Q9 objective 1.3 A dispatcher pastes a consignment id and wants a one-line status back. There is a status tool. No drafting, no multi-step work. Pattern?

- A. Agent loop with BUDGET = 6
- B. Prompt chaining: lookup then format
- C. Augmented LLM: one call with the tool attached
- D. Routing to a status ward

Q10 objective 1.3 Three independent carrier checks, each about 2 seconds, with a 30 second SLA and a daily volume of 1,200. Pattern and the number that justifies it?

- A. Sequential chaining; 6 s is well under 30 s so simplicity wins
- B. Parallel sectioning; wall clock about 2 s instead of 6 s, same token bill
- C. Voting across the three carriers for confidence
- D. An agent that decides which carriers to check

Q11 objective 1.3 KAVI's loop: for turn in range(BUDGET): r = create(...); if r.stop_reason == 'tool_use': run_tools(); elif r.stop_reason == 'end_turn': return text_of(r); else: return text_of(r). Name the defect.

- A. No budget; the loop can run forever
- B. The else returns partial or empty text as a finished answer for max_tokens, refusal, pause_turn and unknown values
- C. text_of should be r.content[0].text
- D. end_turn should also run tools

Q12 objective 1.3 A live response returns stop_reason = 'refusal' on turn 3 of the agent loop. Correct handling?

- A. Append 'Please try again' and call again
- B. Return the text so far as the answer
- C. Escalate to a human with the reason recorded; a refusal is a different path, not a retry
- D. Switch to a bigger model and repeat the same request

Q13 objective 1.3 A reply draft is scored against a written three-item rubric, improves on revision, and the desk accepts at most two revision rounds. Pattern?

- A. Agent loop, so the model decides when it is done
- B. Evaluator-optimizer with max_rounds = 2 and PASS as the exit
- C. Voting: three drafts, pick the majority
- D. A single prompt that asks the model to self-check

Q14 objective 1.3 Turn 2 returns stop_reason = 'pause_turn' from a server-side tool. The loop should:

- A. Treat it as end_turn and return
- B. Append the reply to the transcript and call again, counting the turn against BUDGET
- C. Escalate immediately
- D. Discard the reply and resend the original brief

Q15 objective 1.4 Three subagents each need the complete field notes of the other two before they can report. Which property of the multi-agent design has stopped paying for itself?

- A. Parallelism
- B. Specialised prompts
- C. Context isolation; every report now needs every other's notes, so isolation became re-briefing overhead
- D. Tool restriction

Q16 objective 1.4 In a peer handoff from a triage shift to a resolution shift, the resolution agent re-asks the customer for the consignment id and re-runs the authorization check. The defect?

- A. The resolution agent needs a bigger model
- B. The baton is informal; it should carry findings, context and authorization state as a structured object
- C. Peer handoff is always wrong; use one agent
- D. The triage agent should have resolved the case itself

Q17 objective 1.4 A per-query max_budget_usd is reached while two background subagents are running and a new spawn is requested. Which statement is correct?

- A. Only the new spawn is refused; running subagents finish normally
- B. New spawns are refused, background subagents are stopped, and the query ends
- C. The budget resets for the next layer
- D. The orchestrator switches to a cheaper model and continues

Q18 objective 1.4 An orchestrator requests 25 parallel carrier-check subagents. Concurrency is at its default. What happens, and what is the design response?

- A. All 25 run; the default is 50
- B. 20 run at once and 5 wait; group lanes so fewer, broader subagents are needed, or accept the second wave in the latency budget
- C. The request fails
- D. Depth is exceeded, so the spawn is refused

Q19 objective 1.5 From Meera's brief the subtasks can be listed up front, the list differs per brief, and the shape delegate-then-synthesise never changes. Pattern?

- A. Prompt chaining with a longer chain
- B. Orchestrator-workers: the model writes the plan, code runs it in order, synthesis closes
- C. Re-planning, since the list differs per brief
- D. An agent loop; the model should decide everything

Q20 objective 1.5 Midway, a worker reports CUSTOMS_HOLD, which makes two remaining planned subtasks pointless and calls for two new ones. Which design handles this, and what must it carry?

- A. Orchestrator-workers; the plan is fixed once written
- B. Re-planning after every finding, with a growth cap such as max_new
- C. Routing; send the ticket to the customs ward
- D. Voting; three plans, pick the majority

Q21 objective 1.5 Re-planning code checks the plan after each finding and appends every suggested task with no cap. The defect and the law?

- A. None; re-planning should be unbounded to stay adaptive
- B. The plan can grow without end; Law 3 applies to plans as well as turns
- C. The check should happen once at the end; Law 4
- D. Workers should be bigger models; Law 1

Q22 objective 1.5 Which worker prompt reflects context isolation?

- A. Subtask plus the whole history, the full brief and every SOP, so nothing is missed
- B. Subtask plus only the findings it declared it needs
- C. Subtask only, with no findings at all
- D. The whole plan, so the worker can reorder it

Q23 objective 1.5 What single fact separates prompt chaining from orchestrator-workers?

- A. Chaining uses small models; orchestration uses big ones
- B. In chaining the code writes the step list; in orchestrator-workers the model writes it
- C. Chaining has gates; orchestration has none
- D. Orchestration is always parallel

Q24 objective 1.5 The subtasks for a damage claim are fully known and identical every time. A design pays the model to write the plan for each claim anyway. Reading?

- A. Correct; a model-written plan is more robust
- B. Over-engineering; decomposition in advance belongs in code when the subtasks are known. Dynamic machinery is for uncertainty that survives the brief
- C. Correct, if the planning model is small
- D. Wrong because the plan should be voted on

Q25 objective 1.6 The stem fixes a 30 second SLA on drafts. One option adds three-model voting to every draft, raising p50 latency to 50 seconds 'for accuracy'. Reading?

- A. Choose it; accuracy is the worthier pillar
- B. Reject it; the stated pillar is the contract, and the option optimises a pillar the stem did not fix
- C. Choose it and raise the SLA to 60 s
- D. Reject it because voting never improves accuracy

Q26 objective 1.6 Option G reads 'a future-proof platform that improves customer experience'. How does a 1.6 stem treat it?

- A. As the strongest option, since it covers every pillar
- B. As a wish, not a candidate; a cell that cannot be filled with a number loses to any humbler option that can
- C. As equal to the tiered design
- D. As the cost-pillar winner

Q27 objective 1.6 In a five-pillar matrix, design D maximises efficiency and productivity but breaches cost and SLA; design E meets every target without maximising any. Which is aligned?

- A. D, because it wins two pillars
- B. E; alignment means no pillar breached, not one pillar maximised
- C. Neither; a fourth design is needed
- D. D, if the board accepts the cost

Q28 objective 1.6 Which statement about the transformation pillar is exam-grade?

- A. It cannot be measured, so it is not scored
- B. It is the biggest model's domain
- C. The feedback log of customs-hold patterns becomes a pre-clearance service line; the log is a product, not exhaust
- D. It means replacing dispatchers

Section 10 Sealed key

Do not open until the section is finished.

Section 1 Recall sheets: model answers

Pattern	Tell	Owns the path
Augmented LLM (E1)	one question, tools at hand, no multi-step work	code (one if)
Chaining (E2)	steps identical every time; accuracy over latency	code
Routing (E3)	known categories, different handling each	code
Sectioning (E4)	independent subtasks plus a latency budget	code
Voting (E4)	high-stakes judgement, tolerance for 3x cost	code
Orchestrator-workers (E5)	subtasks unpredictable, but delegate-then-synthesise fixed	code steers, model plans
Evaluator-optimizer (E6)	clear rubric, drafts improve measurably	code
Agent loop (E7)	path itself unknowable; each result steers the next act	model, inside budget + exit
Re-planning (E8)	findings can invalidate the plan mid-run	model plans, code caps growth

stop_reason	Correct handling	The trap
end_turn	natural exit; return partial + text	
tool_use	run every requested tool, append assistant turn + all tool_results, continue	running only the first tool; forgetting the assistant turn
max_tokens	keep partial text, append reply + 'Continue.', continue	returning the truncated text as done
stop_sequence	handled explicitly, usually return or escalate	falls into a swallowing else
pause_turn	append reply, call again; counts against BUDGET	treating as end_turn
refusal	escalate with reason; never a blind retry	'please try again'
context exceeded	escalate or compact; explicit branch	falls into else

Pillars: efficiency first touch 4 h to under 5 min; cost at most Rs 6 per exception; productivity 3x dispatcher throughput, editing not drafting; performance 99 percent of drafts within 30 s; transformation customs-hold pattern reports become a pre-clearance service line. Organs: E6 max_rounds / PASS; E7 BUDGET / end_turn with escalation as floor; E8 the task list empties (run budget) and max_new (growth budget). Caps: spawn depth default 3, concurrency default 20, max_budget_usd refuses new spawns, stops background subagents, ends the query. Stages: input (normalise to ticket schema, validate, scrub PII), processing (router, chains, parallel checks, agent tier with BUDGET), output (drafts for dispatcher edit, re-route orders, human gate on refunds at or above Rs 10,000), feedback (edit diffs, outcome log, weekly eval set tuning router and prompts). The two deleted arcs: the feedback arc and the human gate.

Section 2 Tell-spotting: answers

#	Tell	Pattern	Tempting wrong answer
1	identical steps; accuracy over speed	chaining with gates	agent (OE)
2	one question, nothing else	augmented LLM	chaining (OE)
3	known categories, different handling and model size	routing	one prompt with all wards (VG)
4	independent; 30 s SLA	parallel sectioning	sequential (CM)
5	high stakes; 3x cost accepted	parallel voting	single call (TR)
6	sequence cannot be scripted	agent loop with budget + escalation	chaining with more gates (TR)

#	Tell	Pattern	Tempting wrong answer
7	list differs per brief; shape fixed	orchestrator-workers	re-planning (OE) or chaining (VG)
8	rubric; at most two rounds	evaluator-optimizer	agent (OE)
9	finding makes planned subtasks pointless	re-planning with growth cap	static orchestrator (CM)
10	deterministic rule	plain code, no LLM	router category (OE)
11	category already a field	read the field; no router	router (OE)
12	one prompt, model picks	this is not routing; split into a router + wards	calling it routing (VG)
13	40 ms, hard control loop, structured data	not an LLM seat	small fast model (TR)
14	missing information, human currently asks	Claude drafts the outreach; the customer's reply closes; a gate	full automation (HL)

Section 3 Predict the output: answers

3.1 Banner line; '# turn 1 stop_reason=tool_use (get_shipment_status); # turn 2 stop_reason=end_turn'; '{"category": "customs", "urgency": "high"}'. Without the tool there is one turn ending end_turn and the category would most likely be 'delay', because the email never says customs.

3.2 gate() raises ValueError('bad category: delay.') and the line stops. That is the gate doing its job: it rejected a string not on the list. The fix belongs in classify: strip punctuation, for example .strip('! ').lower(), or tighten the prompt.

3.3 Sectioning about 2.4 s (the slowest), sequential about 5.3 s (the sum), 2.9 s saved. Token bill unchanged.

3.4 PASS on round 1: draft + editor = 2 calls. Criticism both rounds: draft, editor, revise, editor, revise = 5 calls; the second revision is returned because the budget is spent. The loop never runs a third round.

3.5 turn 1 tool_use (get_shipment_status); turn 2 tool_use (find_alternate_carriers); turn 3 end_turn. Three paid calls. Maximum ever: 6, then escalation.

3.6 turn 3 max_tokens; the partial text is kept and 'Continue.' appended; turn 4 end_turn; the returned text is partial plus the rest. A loop treating max_tokens as done would have returned half a recommendation.

3.7 'ESCALATED to Meera's desk (budget exhausted): Why is KF-2481 stuck, and what are our options?' The word is the reason in brackets. A hand-off with a reason is a designed outcome; one without is a failure.

3.8 run 1 status check; replan drops 3 and adds ids 4 and 5; run 2 carrier options; run 4 file e-way bill; run 5 notify Padma; 'grew=2, budget max_new=3 not exhausted'. With MAX_NEW = 1 only 'file e-way bill' is added as id 4, grew = 1, and the budget line says EXHAUSTED.

Section 4 Red pen: answers

#	Defects	Law	Fix
4.1	while True has no budget; the else treats every non-tool_use stop as finished; content[0].text breaks when a thinking block is first	3	for turn in range(BUDGET); one branch per stop_reason; text_of() by block type
4.2	refusal is retried blindly; the else swallows max_tokens, pause_turn and unknown values	3	refusal escalates with reason; max_tokens continues with partial kept; else escalates 'unhandled'
4.3	assistant turn not replayed (amnesia); tool_result has no tool_use_id	cell rule	append {'role': 'assistant', 'content': msg.content} first; add 'tool_use_id': call.id
4.4	no growth cap, so the plan can grow forever; added tasks are bare strings with no id or needs; nothing is dropped	3, 4	grew < max_new check, clip adds, build task dicts with fresh ids, apply drop
4.5	no gate on the category; SPECIALISTS[cat] raises KeyError on an unmapped word	2	gate(classify(email)); SPECIALISTS.get(cat, SPECIALISTS['delay'])
4.6	voting on a delay apology: 3x cost for no judgement content; big model for a trivial task	1, 5	vote only on refunds and other money decisions; small model for apologies
4.7	worker given the whole history, the full brief and all SOPs: context isolation lost, cost up, confusion up	4	pass only the findings in sub['needs']

#	Defects	Law	Fix
4.8	evaluator and optimizer collapsed into one prompt; no rubric, no rounds, no PASS; nothing measurable	3 (no budget, no exit)	separate editor call with a written rubric, max_rounds, PASS exit

Section 5 Arithmetic: answers

#	Working	Answer
5.1	$0.80 \times 1.50 + 0.15 \times 6.00 + 0.05 \times 28.00 = 1.20 + 0.90 + 1.40$	Rs 3.50 avg; Rs 4,200 a day; meets the Rs 6 pillar
5.2	$1,200 \times 22 = 26,400$; $26,400 - 4,200$	Rs 22 avg, breach; Rs 26,400 a day; Rs 22,200 more; OE
5.3	$\max(1.8, 2.4, 1.1)$ vs $1.8 + 2.4 + 1.1$	2.4 s vs 5.3 s; 2.9 s saved; token bill identical
5.4	108 refunds: $108 \times 4 = 432$ vs $108 \times 12 = 1,296$. Delays: $660 \times 0.80 \times 2$ extra calls	Rs 432 vs Rs 1,296 a day; voting on apologies adds Rs 1,056 a day and breaches cost for nothing
5.5	6×3.20 ; $3,600 \text{ s} / 2 \text{ s} = 1,800$ calls $\times 3.20$	Rs 19.20 worst case; Rs 5,760 for the unbounded hour
5.6	5 percent of 1,200 = 60 slow drafts; $1,140 / 1,200$	95 percent within 30 s; SLA breached; agent tier may take at most 1 percent (12 drafts)
5.7	$3 + 3 = 6$ tasks; $6 \times (2 + 4) + 4 + 4$	6 tasks; Rs 44 maximum; without the cap there is no maximum
5.8	$30 \times 3 = 90$; $90 \times 40 = 3,600$	90 a day each; 3,600 capacity; 3x the 1,200 load
5.9	$\text{ceil}(50 / 20) = 3$ waves $\times 6 \text{ s}$	18 s minimum; 3 waves
5.10	layer 4 exceeds depth 3	the translator spawn is refused; fleet-scale Law 3

Section 6 Architecture completion: answers

#	Missing	Stage	Family
1	normalisation into one ticket schema with PII scrubbing; three prompts per channel is three pipelines	input	CM
2	human gate on refunds at or above Rs 10,000; auto-send for every amount is the wrong output control	output	HL
3	the feedback arc: edit diffs and outcomes into a weekly eval set; without it launch quality is forever	feedback	SF
4	a default route to a human queue for 'other', counted and reviewed to fix the category set; biggest model is the wrong fix	processing to output	OE
5	a logged, counted default route to a human queue with the ticket intact	output	HL

Section 7 Multi-agent: model answers

7.1 Context isolation has stopped paying; every report needs every other's field notes, so isolation is now re-briefing overhead. Merge into a single agent, or re-cut the subtasks so they are genuinely separable.

7.2 An informal baton. The handoff should be a structured object carrying findings, context and authorization state, so phase two never re-asks or re-authorizes.

7.3 Only reports travel back; field notes stay inside each subagent's own context. The orchestrator holds six summaries, not six transcripts.

7.4 New spawns are refused, background subagents are stopped, the query ends. Fleet-scale Law 3.

7.5 Symptoms: context bloats, one prompt serves three expertises, tools multiply past a handful. Split under Law 1 and Law 2: specialised prompts and tool restriction per subagent.

7.6 Peer handoff. Two clear phases where phase two needs findings and authorization state, not raw notes, is a baton, not a tree. Orchestrator plus subagents is for parallel, separable subtasks reporting to one synthesiser.

Section 8 Pillar matrix: answers

Pillar	D one agent	E tiered	F rules only
Efficiency at most 5 min	3 min met	1 min met	4 h breached
Cost at most Rs 6	Rs 24 breached	Rs 3.90 met	Rs 0 met
Productivity 3x	about 2.8x, near but editing drafts takes longer; mark as met or borderline	about 3x met	1x breached
Performance 99 percent under 30 s	p50 38 s so well under 99 percent; breached	99.2 percent met	n/a, no drafts
Transformation	logs exist, unmined	feedback log becomes pre-clearance line	none

8.1 E. Alignment means no pillar breached, not one pillar maximised. 8.2 G has no cell that can be filled with a number; it is a wish, not a candidate. The unpriced-option trap. 8.3 'D costs Rs 24 against a Rs 6 ceiling and misses the 30 second SLA on most drafts; E meets all five targets at Rs 3.90 and 99.2 percent. Capable but misaligned loses to aligned.'

Section 9 Twenty-eight questions: key

Q	Obj	Answer	The tell	Wrong options by family
1	1.1	B	tell: 70 percent detectable from one field	A OE, C TR, D CM
2	1.1	A	tell: issue the order is a paid irreversible action	B VG, C HL, D OE
3	1.1	B	tell: measure, number, horizon, volume	A VG, C OE, D SD
4	1.1	B	tell: already types the category	A TR, C OE, D OE
5	1.2	B	tell: auto-sent for all refund amounts	A OE, C TR, D OE
6	1.2	C	tell: feedback, not processing	A B D are processing signals; VG
7	1.2	B	tell: one pipeline, defects cluster at the channel boundary	A SD, C OE, D CM
8	1.2	B	tell: 20 percent 'other' is a category-set signal	A OE, C SF, D SF
9	1.3	C	tell: one question, tool at hand, nothing else	A OE, B OE, D VG
10	1.3	B	tell: independent plus SLA; max not sum	A CM, C VG, D OE
11	1.3	B	tell: the else	A false (budget exists), C VG, D VG
12	1.3	C	tell: refusal is a path, not a retry	A SF, B SF, D TR
13	1.3	B	tell: rubric, measurable, at most two rounds	A OE, C TR, D VG
14	1.3	B	tell: pause_turn resumes; still a turn	A SF, C TR, D SF
15	1.4	C	tell: every report needs every other's notes	A B D VG
16	1.4	B	tell: re-asks and re-authorizes	A TR, C OE, D CM
17	1.4	B	tell: the three documented effects	A C D VG
18	1.4	B	tell: default 20	A VG, C VG, D VG
19	1.5	B	tell: list differs per brief, shape fixed	A VG, C OE, D OE
20	1.5	B	tell: finding invalidates remaining plan	A CM, C VG, D VG
21	1.5	B	tell: no cap on growth	A SF, C TR, D CM
22	1.5	B	tell: declared needs only	A CM, C CM, D VG
23	1.5	B	tell: who writes the list	A C D VG
24	1.5	B	tell: fully known, identical	A OE, C OE, D VG
25	1.6	B	tell: the stated pillar is the contract	A TR, C CM, D VG
26	1.6	B	tell: no number	A SD, C VG, D VG
27	1.6	B	tell: no breached cell	A TR, C SD, D CM
28	1.6	C	tell: the log is a product	A SD, B VG, D VG

Scoring by objective: 1.1 Q1 to 4, 1.2 Q5 to 8, 1.3 Q9 to 14, 1.4 Q15 to 18, 1.5 Q19 to 24, 1.6 Q25 to 28.
Anything under four of six on 1.3 or 1.5 sends you back to E7 and E8 in d1/live, line by line.

Section 11 Miss ledger

One row per miss, across every section. The family column is the one that matters: three rows under the same family is a habit, not a slip. Finish by writing the one sentence you will read before the exam.

Section / Q	Objective	My answer	Correct	Family	The tell I skipped

The one sentence I will read before the exam:

Where the material lives

Runnable exhibits: d1/e1_augmented_11m.py to d1/e8_replanning.py with DRY_RUN=1, and ./d1/run_all.sh. Plain live code: d1/live/. Explainers: D1_Explained_COMBINED_All_8_Parts.pdf, D1_Live_Code_With_Flowcharts.pdf. The forty original questions and the Atlas: CCAR-P_D1_Control_Tower_Brief.pdf.